

Technical Report

louis-rs

Security Assessment

Prepared for:
OSTIF



SHIELDER
WEB SECURITY

1. Document Details

Classification	Public - CC BY-SA 4.0 License
Last review	September 29, 2026
Author	Pietro Tirennà, Nicolò Daprelà

1.1. Version

Identifier	Date	Author	Note
v1.0	April 20, 2026	Pietro Tirennà, Nicolò Daprelà	First version
v1.1	April 22, 2026	Abdel Adim Oisfi	Peer review
v1.2	September 29, 2026	Pietro Tirennà, Nicolò Daprelà	Public release

1.2. Contacts Information

Company	Name	Position	Contact
Shielder	Abdel Adim `Smaury` Oisfi	CEO	abdeladim.oisfi@shielder.it
Shielder	Pietro Tirennà	Security Researcher	pietro.tirennà@shielder.it
Shielder	Nicolò Daprelà	Security Researcher	nicolo.daprela@shielder.it
OSTIF	Derek Zimmer	Executive Director	derek@ostif.org
OSTIF	Amir Montazery	Managing Director	amir@ostif.org
OSTIF	Helen Woeste	Communications and Community Manager	helen@ostif.org
OSTIF	Tom Welter	Project Manager	tom@ostif.org
SBS	Christian Egli	louis-rs Core Developer	christian.egli@sbs.ch

1.3. *About OSTIF*

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our [Github](#).

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



2. Summary

1. Document Details	2
1.1. Version	2
1.2. Contacts Information	2
1.3. About OSTIF	3
2. Summary	4
3. Executive Summary	5
3.1. Overview	5
3.2. Context and Scope	5
3.3. Methodology	6
3.4. Audit Summary	7
3.5. Results Summary	8
3.6. Findings Severity Classification	9
3.7. Remediation Status Classification	10
4. Fuzzing	11
4.1. Setup	11
4.2. Text Translation	11
4.3. Table Parsing	11
4.4. Results	12
4.5. Deployment	12
5. Recommendations	13
5.1. Restrict Table Loading to Trusted Sources	13
5.2. Setup Dependency Scanning	13
5.3. Setup CodeQL Scanning	14
6. Findings Details	15
6.1. Denial of Service via Uncontrolled Recursion in Table Inclusion	15
6.2. Denial of Service via Regex Catastrophic Backtracking on Custom Tables	17
6.3. Denial of Service via Panic on Double Negation in Match Rule Patterns	19
7. Hardening	21
7.1. Support Configurable Translation Timeouts	21
7.2. Restrict File Access in the include Opcode	21

3. Executive Summary

The document aims to highlight the findings identified during the “Security Assessment” against the “louis-rs” product described in section “3.2 Context and Scope”.

For each detected findings, the following information are provided:

- **Severity:** findings score (“3.6 Findings Severity Classification”).
- **Affected resources:** vulnerable components.
- **Status:** remediation status (“3.7 Remediation Status Classification”).
- **Description:** type and context of the detected finding.
- **Impact:** loss of confidentiality, data integrity and/or availability in case of a successful exploitation and conditions necessary for a successful attack.
- **Proof of Concept:** evidence and/or reproduction steps.
- **Suggested remediation:** configurations or actions needed to mitigate the finding.
- **References:** useful external resources.

3.1. Overview

From March 16, 2026 to April 20, 2026 Shielder was hired by the Open Source Technology Improvement Fund (OSTIF) to perform a Security Audit of louis-rs, a rewrite in Rust of liblouis, a C open-source library for braille translation, back-translation and formatting.

The goal of the rewrite is to greatly reduce the attack surface of the library by leveraging Rust's compile-time safety and robust memory management, since (quoting) *"many if not most of the CVEs of liblouis are rooted in the manual memory management in the C version of liblouis"*.

A team of 2 (two) Shielder researchers worked on this project.

3.2. Context and Scope

louis-rs is built around the following core components:

- **CLI:** the `louis` binary, which provides commands to parse braille tables, translate text to/from braille, trace rule application, run YAML-based test suites, and query table metadata.
- **Parser:** the module that reads and expands liblouis braille table files.
- **Translator:** the translation pipeline that compiles parsed rules into lookup structures and executes them in stages to convert text to braille (forward-translation) or braille to text (back-translation).
- **Braille tables:** liblouis-format `.ctb/.utb` files that define the character mappings and contraction rules for a given language. These are resolved from the filesystem via the `LOUIS_TABLE_PATH` search path.

For what concerns the attack surface in scope, the researchers – with the support of core maintainers – have modeled the following main attack scenarios:

- A malicious or malformed braille table file, supplied by an attacker or fetched from an untrusted source, is loaded by the parser. Crafted table content might trigger memory safety issues, excessive resource consumption, or unexpected behavior during parsing or rule compilation.
- An attacker-controlled input string is passed to the translator for conversion. Crafted input might exploit edge cases in trie lookups, regexp matching, or multipass rule execution to cause crashes, infinite loops, or incorrect output.
- An attacker able to inject code in the supply chain of the project, either in development or deployment/distribution, might compromise end-users of the library.

3.3. *Methodology*

The activity was carried out following the standard Shielder security audit methodology, which focuses on discovering:

- Vulnerabilities: findings that can be immediately exploited by an attacker to compromise the CIA (confidentiality, integrity, availability) triad of the system.
- Hardening recommendations: properties/patterns that, if implemented, would reduce the attack surface of the system.

Due to the typically constrained resources – in terms of time and financials – of Open Source projects, it is critical to avoid flooding maintainers with theoretical findings, and rather focusing on actionable and reasonable issues that, upon remediation, immediately increase the security posture of the project.

For this audit, the following techniques and approaches were employed:

- Collaborative threat modeling, with the goal of identifying the critical parts of the attack surface.
- Manual source code review, focused on the most exposed component – such as input forward-translation.
- Static Analysis using dedicated SAST tools, such as [semgrep](#) and [CodeQL](#).
- Fuzzing of the most "complex" bits, namely text-braille translation and table parsing.
- AI-assisted source code review, focused on reducing the time needed to understand the code, or quickly explore less-critical portions of the codebase to increase the audit coverage.

3.4. *Audit Summary*

The overall security posture of louis-rs is adequately mature from both a secure coding and design point of view, even though there is still room for further refinement in some areas.

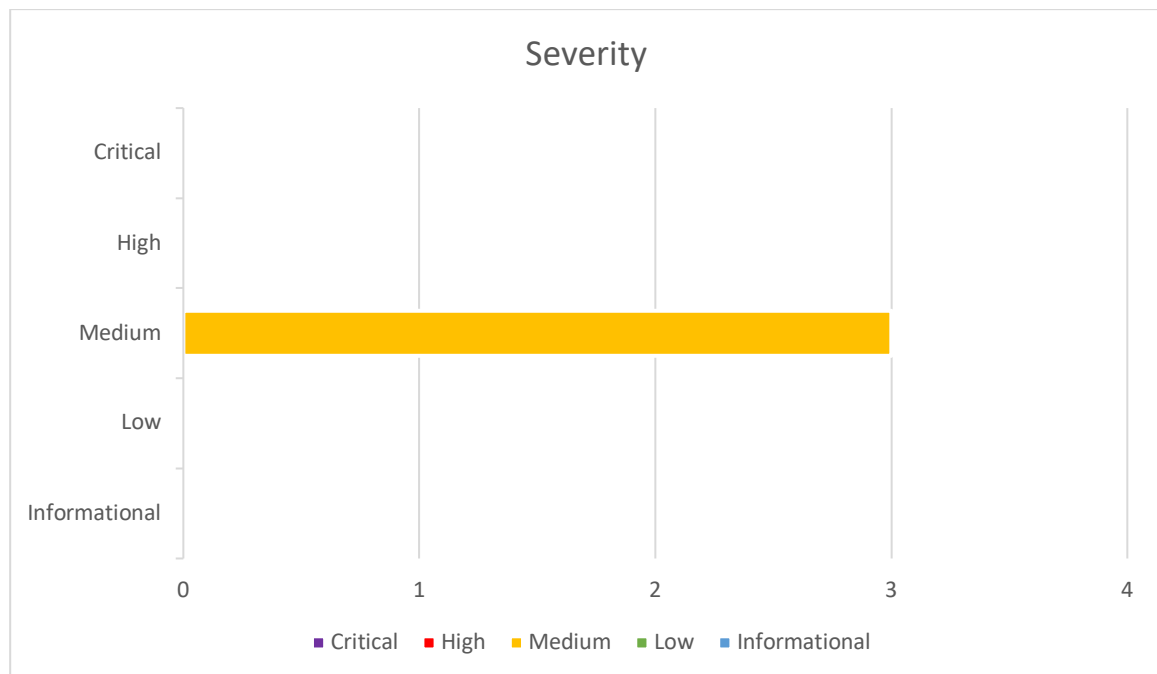
The Shielder team discovered a total of three (3) vulnerabilities with a Medium severity. Additionally, the audit produced two (2) hardening recommendations.

The main threats are caused by some denial of services when parsing custom tables, and an unmaintained dependency in the supply chain which might be used to compromise end-users.

It is important to mention that, at the time of writing, louis-rs is in an Alpha state, therefore the code will likely be subject to many changes in the future.

3.5. Results Summary

The following chart shows the number of findings found per severity:



ID	Finding	Severity	Status
1	Denial of Service via Uncontrolled Recursion in Table Inclusion	MEDIUM	Closed
2	Denial of Service via Regex Catastrophic Backtracking on Custom Tables	MEDIUM	Closed
3	Denial of Service via Panic on Double Negation in Match Rule Patterns	MEDIUM	Closed

3.6. Findings Severity Classification

Severity	Description
CRITICAL	Vulnerability that allows to compromise the whole application, host and/or infrastructure. In some cases, it allows access, in read and/or write, to highly sensitive data, totally impacting the resources in terms of confidentiality, integrity and availability. Usually, such vulnerabilities can be exploited without the need of valid credentials, without considerable difficulty and with the possibility of automated, highly reliable, and remotely triggerable attacks. Vulnerabilities marked with this severity must be resolved quickly, especially in production environment.
HIGH	Vulnerability that significantly affects the confidentiality, integrity, and availability of confidential and sensitive data. However, the prerequisites for the attack affect its likelihood of success, such as the presence of controls or mitigations and the need of a certain set of privileges.
MEDIUM	Vulnerability that allows to obtain only a limited or less sensitive set of data, partially compromising confidentiality. In some cases, it may affect the integrity and availability of the information, but with a lower level of severity. In addition, the chances of success of such vulnerability may depend on external factors and/or conditions outside the attacker's control.
LOW	Vulnerability resulting in a limited loss of confidentiality, integrity, and availability of data. In some cases, it depends on conditions not aligned to a real scenario or requires that the attacker has access to credentials with a high level of privileges. In addition, a low severity vulnerability may provide useful information to successfully exploit a higher impact vulnerability.
INFORMATIONAL	Problems that do not directly impact confidentiality, integrity, and availability. Usually, these problems indicate the absence of security mechanisms or the improper configuration of them. Mitigation of this type of problem increases the general level of security of the system and allows in some cases to prevent potential new vulnerabilities and/or limit the impact of existing ones.

3.7. Remediation Status Classification

Status	Description
Open	Vulnerability not mitigated or insufficient mitigation.
Not reproducible	Vulnerability not reproducible due to environment changes or to mitigation of other vulnerabilities required in the reproduction steps.
Closed	Vulnerability mitigated. The security patch applied is reasonably robust.

4. Fuzzing

One of the goals of the audit was to develop a set of fuzzers to dynamically stress some of the most complex parts of the library: table parsing and text translation.

This section documents how the fuzzers were designed and developed, and how they can be integrated in the CI/CD pipeline of the project.

4.1. *Setup*

After assessing the available options, rust-fuzz was selected due to its integration with the Rust ecosystem and ease of use.

However, the Cargo.toml of the fuzzer crate was patched so to use the [libafl libfuzzer](#) ship instead of the default libfuzzer runtime. By doing this, the fuzzer can automatically use the built-in [grimoire mutator](#), which enables transparent, automatic structure-aware fuzzing of the translation table syntax.

4.2. *Text Translation*

Translation of text to braille is arguably the most exposed and critical attack surface. In order to fuzz the text that is translated by the library, it is necessary to supply translation tables. Too simple a table might leave large portions of the translation engine unreachable, while a single real-world table would only exercise the opcode subset that language happens to use. To address this, the fuzzer uses a hybrid table (fuzz_maximal.ctb) that layers six real upstream (extracted from the [liblouis](#) project) tables spanning different languages – English grade 2, UEB grade 1, French, Spanish, Uzbek, and Arabic – with a synthetic overlay containing short ASCII triggers for opcode families that would otherwise require specific character sequences to reach, such as match, context, multipass rewrites, and swap-backed captures. An LLM was adopted to extract the subset of upstream tables and generate the synthetic rules.

The seed corpus consists of hand-crafted text inputs covering common translation scenarios: plain ASCII, punctuation, numeric sequences, contractions, Unicode, and mixed-case text. A dictionary is also provided to the fuzzer, including the tokens that trigger the rules in the table, allowing the engine to immediately reach meaningful inputs without having to rediscover them.

4.3. *Table Parsing*

This fuzzer generates arbitrary table source code and, if the table compiles successfully, exercises the resulting translator against a fixed set of representative sample strings in both forward and backward directions.

The public API only accepts tables as filesystem paths. Since the fuzzing loop generates input bytes, this would have meant, for each iteration, writing to a temporary file and passing the path to the API, wasting a significant amount of time on I/O operations. To overcome this issue, a simple public API was added to build a table directly from in-memory data.

The seed corpus is structured in two layers. The first consists of synthetic minimal tables, each exercising a single opcode family. A small set of intentionally invalid tables is also included to seed the fuzzer into error-handling paths from the start. The second layer imports every table from the upstream liblouis project with include directives stripped, since the fuzzer operates, as mentioned, without filesystem access. This provides hundreds of real-world tables as additional starting points.

As with the translate fuzzer, a domain-specific dictionary is provided, listing every opcode keyword, common dot patterns, special characters, and representative UTF-8 byte sequences including braille codepoints. This allows the mutation engine to compose syntactically meaningful table lines rather than relying on random byte mutation alone.

As mentioned in the “Setup” section, the `libaf1_libfuzzer` shim used in place of the deprecated `libfuzzer-sys` automatically enables the Grimoire mutator, a coverage-guided mutation strategy that infers structural patterns from the input corpus and recombines them to produce new inputs. This is particularly effective for the table fuzzer, where the input is a structured, line-oriented text format: Grimoire can learn recurring syntactic fragments – such as opcode-dots pairs or modifier prefixes – and compose them into structurally plausible tables, increasing the ratio of valid input data.

4.4. Results

The Shielder team ran the fuzzers in a dedicated VPS for a couple of days. As a result, the finding **6.3 - Denial of Service via Panic on Double Negation in Match Rule Patterns** was discovered.

4.5. Deployment

The Shielder team proposed the code of the aforementioned fuzzers in the following Pull Request:

<https://github.com/liblouis/louis-rs/pull/7>

The code comes together with documentation on the choices adopted for the harnesses and the steps needed to run a fuzzing process.

Due to the fact that `louis-rs` is currently in alpha state, it would likely not satisfy the [requirements for OSS-Fuzz](#). Nonetheless, the fuzzers were developed in a way that is fully compatible with the [documentation](#) provided by OSS-Fuzz on how to deploy Rust projects, so to simplify the future integration of `louis-rs` on Google’s fuzzing infrastructure.

5. Recommendations

5.1. *Restrict Table Loading to Trusted Sources*

Due to the flexibility of the syntax – for instance, by abusing regular expression syntax – it is possible to craft a custom table in a way that using it to translate causes a Denial of Service (e.g. resource starvation or hang). Therefore, it is recommended to document that loading translation tables is a privileged operation: only trusted users should be able to supply or modify table files. This should be explicitly stated in the library's API documentation and in any integration guide, so that downstream consumers do not inadvertently expose table loading to unprivileged input.

5.2. *Setup Dependency Scanning*

At the current state the project is lacking some best practices related to SBOM management, specifically:

- There is not an integration with SBOM generation tools.
- There is no automated dependency vulnerability scanning in the build process.
- There is no dependency license compliance checking beyond the basic license declaration.

It is advised to integrate proper jobs in the CI/CD pipeline to automate the process of SBOM generation, dependency scanning and license checking.

The Shielder team implemented a proposal solution in the following PR: <https://github.com/liblouis/louis-rs/pull/6>

By running the dependency scanners against the project, the team identified seven outdated dependencies:

- clap
- enumset
- env_logger
- hyphenation
- log
- tabled
- thiserror

Moreover, the team also identified an unmaintained third-party dependency:

- bincode

5.3. *Setup CodeQL Scanning*

GitHub offers the possibility to easily integrate the analysis offered by CodeQL inside the CI/CD pipeline using workflows.

It is recommended to integrate CodeQL scanning in the CI/CD pipeline of the project.

The Shielder team implemented a proposal solution in the following PR:
<https://github.com/liblouis/louis-rs/pull/6>

6. Findings Details

Analysis results are discussed in this section.

6.1. Denial of Service via Uncontrolled Recursion in Table Inclusion

Severity	MEDIUM
Affected Resources	src/parser.rs
Status	Closed

August 13, 2026 Update

The vulnerability was fixed with the [linked change](#), by rejecting circular or excessively nested inclusion chains.

Description

The include opcode allows a table to pull in rule definitions from another file. When processing includes, the parser recursively expands each referenced table without:

1. **Cycle detection** – a table can include itself (directly or transitively), causing infinite recursion.
2. **Depth limiting** – even without cycles, a sufficiently deep chain of includes will exhaust the call stack.

Either condition results in a stack overflow and crashes the program.

Impact

An attacker who can supply or modify a translation table can crash the program via stack overflow, causing a denial of service.

Attack Complexity

The attacker must be able to control the translation table passed to the library, or modify any table file in the inclusion chain.

Related Issues

N/A

Proof of Concept

1. Create a self-referencing table:

```
echo "include a.utb" > a.utb
```
2. Run the parser:

```
louis parse a.utb
```

3. The program crashes with a stack overflow.

Suggested Remediations

Implement both:

- **Cycle detection** during parsing, to reject circular includes with a clear error message.
- **A recursion depth limit**, to gracefully halt execution when the inclusion chain exceeds a reasonable threshold – covering both cyclic and non-cyclic stack exhaustion.

References

- <https://cwe.mitre.org/data/definitions/674.html>

6.2. Denial of Service via Regex Catastrophic Backtracking on Custom Tables

Severity	MEDIUM
Affected Resources	src/translator/regexp.rs
Status	Closed

August 13, 2026 Update

The vulnerability was fixed with the [linked change](#), by replacing the backtracking logic with an implementation of PikeVM.

Description

Some of the rules handled by the `louis-rs` translator use regular expressions. One example is the `match` opcode, which is used to define patterns to match and replace markers with a proper translation.

`louis-rs` includes a custom implementation to handle regular expressions, based on a Regular Expression Virtual Machine.

The implemented virtual machine does not verify how much backtracking it performs, being thus susceptible to DoS attacks with maliciously crafted regular expressions.

Impact

An attacker could include a malicious regular expression, crafted to cause catastrophic backtracking, in a custom table designed for `louis-rs`. By subsequently providing a text designed to trigger the backtracking in the regular expression, it might be possible to crash the program using the library.

Attack Complexity

The attacker must be able to control one of the tables passed to the program, or at least inject rule definitions.

Alternatively, the attacker would need to review the regular expressions contained in the rules bundled with a specific program, as they might be able to find unintentionally vulnerable patterns. It should be highlighted that all the tables included in `liblouis` – the C implementation – have been analyzed for similar patterns and none were found.

Furthermore, the attacker would need a way to control some text that is parsed by the program to trigger the backtracking.

Related Issues

N/A

Proof of Concept

1. Create a table containing a vulnerable regular expression, for instance run the command:

```
echo 'match (a+)+ b b 1' > redos.utb
```
2. Run the translator using the following command:

```
louis translate redos.utb aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```
3. In a separate terminal run the command:

```
top
```
4. Notice that the louis process is consuming a lot of CPU while it is stuck doing backtracking.

Suggested Remediations

It is advised to add a limit to the level of backtracking which can be performed during a translation to gracefully end attempts which go beyond the threshold.

References

- <https://cwe.mitre.org/data/definitions/674.html>
- <https://snyk.io/blog/redos-and-catastrophic-backtracking/>

6.3. Denial of Service via Panic on Double Negation in Match Rule Patterns

Severity	MEDIUM
Affected Resources	src/translator/regexp.rs
Status	Closed

August 13, 2026 Update

The vulnerability was fixed with the [linked change](#), by rejecting double negations in match/context patterns.

Description

The match opcode in `louis-rs` tables accepts regular expression patterns for pre-match and post-match contexts. These patterns support a negation operator (`!`), which inverts the meaning of the following sub-pattern (e.g., `!a` matches anything that is not `a`).

The pattern parser allows arbitrary nesting of the negation operator, producing AST nodes like `Negate(Negate(X))`. When converting from the parsed Pattern AST to the Regexp AST, each `Pattern::Negate` layer calls `Regexp::negate()` on the inner expression.

However, `Regexp::negate()` handles the "positive-to-negative" conversion (e.g., `String` → `NotString`, `CharacterClass` → `NotCharacterClass`) but marks the reverse "negative-to-positive" cases as unreachable!(). When a double negation produces a `NotString` or `NotCharacterClass` variant and `.negate()` is called on it a second time, the program panics.

Impact

An attacker could craft a table containing a match rule with a double-negated pattern (e.g., `!!foo` in the pre or post expression). When the table is loaded to be used for translation, the program panics, causing a denial of service.

Attack Complexity

The attacker must be able to control one of the tables passed to the program, or at least inject rule definitions containing match opcodes. No specific input text is required – the panic occurs during table compilation, before any text is translated.

It should be highlighted that all the tables included in `liblouis` – the C implementation – have been analyzed for similar patterns and none were found.

Related Issues

N/A

Proof of Concept

1. Create a table containing a double-negated match pattern:

```
echo 'match !!a b c 1' > crash.utb
```

2. Run the translator:

```
louis translate crash.utb "any text"
```

3. The program panics with:

```
thread 'main' panicked at 'internal error: entered unreachable code',  
src/translator/regex.rs:85
```

Suggested Remediations

Handle double negation in `Regex::negate()` by inverting `Not*` variants back to their positive form:

- `NotString(s).negate() → String(s)`
- `NotCharacterClass(c).negate() → CharacterClass(c)`
- `NotVariableEqual(i, v).negate() → VariableEqual(i, v)`

Alternatively, double negation could be rejected at parse time in `PatternParser::negate()` by checking whether the inner pattern is already a `Negate` and returning a parse error.

References

- <https://cwe.mitre.org/data/definitions/617.html>

7. Hardening

7.1. *Support Configurable Translation Timeouts*

As a defense-in-depth measure, the library could expose a configurable timeout for translation operations, allowing downstream consumers to set a configurable timeout and abort translations that exceed it. This provides a simple safety net against denial-of-service when processing untrusted input.

7.2. *Restrict File Access in the include Opcode*

As a defense-in-depth measure, the library could restrict the file paths accepted by the include opcode to prevent directory traversal and arbitrary file reads. Currently, an include directive with a relative path containing dot-dot-slash (..) sequences can traverse outside the intended search path directories, and an absolute path bypasses the search path entirely. It is instead recommended to validate that resolved paths remain within the configured search path boundaries to protect against unintended side effects of parsing untrusted tables.